Classroom Survey Web App - Development Summary

How I Used the Specifications

I started the CSE 4095 Classroom Survey Web App by organizing the instructor-provided specification, database, API, UI, and TODO documents alongside the assignment instructions. Instead of asking Codex to produce the entire application at once, I first directed it to review the requirements, identify inconsistencies, and prepare an implementation plan. For example, we standardized the public student route as /s/:slug and documented how drafts, active surveys, closed surveys, and archived surveys should behave. I also asked the agent to preserve the original specifications and distinguish verified work from incomplete tasks.

AI-Generated Implementation

I used Codex as a coding assistant throughout the development process. It generated and refined the React, TypeScript, Vite, and Tailwind application structure; instructor login and dashboard; survey creation and editing; QR presentation page; mobile student survey; results visualizations using Recharts; CSV export; and tests. I guided the work in separate phases, reviewed its explanations, supplied actual error messages, and requested targeted corrections. The project was not simply a single generated output: features were implemented, checked, debugged, and revised incrementally.

Database Configuration and Communication

The application uses Supabase PostgreSQL with five related tables: surveys, questions, choices, responses, and answers. Codex prepared versioned migrations with primary and foreign keys, checks, indexes, Row Level Security (RLS), and database functions. Instructor access is based on a trusted Supabase Auth app_metadata.role value of instructor, not merely having an account. The React frontend uses the Supabase JavaScript client with the project URL and publishable key. It invokes database RPCs to save surveys, activate or close them, submit anonymous responses atomically, and retrieve instructor-only results. The database validates submissions and limits unauthorized reads and writes; private service-role credentials are not embedded in browser code.

Problems, Debugging, and Resolution

The debugging process was an important part of my work. When the login form remained disabled after I configured Supabase, I asked Codex to inspect the application rather than assume the credentials were wrong. The cause was an intentionally disabled placeholder from the foundation phase, and Codex replaced it with real authentication and route protection. When saving a survey failed, the remote API returned PGRST202 because save_survey was unavailable in its schema. I used the reported diagnosis to review the necessary SQL migration and API schema refresh steps. I also reported missing results and duplication functions, and a QR code that would not open on my phone because a local development URL used localhost. The latter prompted a configurable

public application URL. These cases showed why real environment testing matters even when automated tests pass.

Deployment, Verification, and Learning

I created a GitHub repository, made a Phase 1 commit, and continued using Git to manage the application. The recorded production audit found the application publicly reachable through a Cloudflare-hosted HTTPS address, with route refreshes functioning and anonymous access restrictions working in the tested cases. The documented local checks included 67 passing unit/service tests, 88 passing Playwright tests, TypeScript, linting, formatting, production build, and disposable PostgreSQL tests. However, the production audit explicitly left several authenticated workflows, real student submissions, historical results, CSV compatibility, and physical-phone QR testing for manual verification. The main lesson I learned was that AI-assisted full-stack development still requires understanding data flow, database permissions, deployment configuration, and the difference between generated code, local evidence, and a fully verified live system.